Windy Gridworld with n-step Sarsa and Sarsa(λ)

A Code-Centric Technical and Mathematical Analysis

Publication-Oriented Report for a Research Software Portfolio

Contents

Executive Summary

The supplied source implements a tabular reinforcement-learning study of the classic Windy Gridworld task, comparing three configurations of n-step Sarsa ($n \in \{1, 5, 10\}$) with four configurations of Sarsa(λ) ($\lambda \in \{0, 0.1, 0.5, 1\}$), followed by a focused learning-rate study. The implementation is organized as a reproducible experiment pipeline: configuration is centralized in an immutable `dataclass`; the environment is represented explicitly; training kernels are accelerated with Numba; per-episode outcomes are stored in Pandas tables; JSON artifacts record configuration and environment details; and a packaging routine collects the generated artifacts. The source itself identifies the task as the Windy Gridworld from Sutton and Barto and describes an episode as receiving a reward of -1 per time step until the goal is reached.

The report intentionally focuses on the implementation, methodology, mathematical formulation, execution flow, and reliability considerations rather than reproducing figures or experimental outputs. This reflects the scope of the deliverable: a code-explanation report rather than an experimental-results paper. The report therefore does not include the plots that the program is capable of generating.

A key audit finding is that the uploaded .py artifact is not, as saved, a clean directly executable Python module: the file begins with a notebook-oriented documentation block before the `from __future__ import annotations` statement. Python requires future imports to occur before ordinary executable content. The report consequently distinguishes the intended computational design from the exact executable state of the uploaded artifact. A second set of important implementation observations concerns experimental aggregation: the configured main run has 100 episodes while the default summary window is also 100 episodes, so the reported final-window mean spans the entire main run; the alpha study has only 80 episodes, so its nominal final 100-episode window also spans the entire run. These are properties of the code, not inferred author intentions.

1 Introduction

Reinforcement learning studies how an agent learns a policy through interaction with an environment, receiving rewards after actions and updating value estimates from experience. The uploaded project uses a deliberately small tabular environment so that the learning algorithms can be implemented directly rather than hidden behind a high-level reinforcement-learning framework. This makes the source especially useful for demonstrating algorithmic understanding: state indexing, epsilon-greedy action selection, multi-step return construction, eligibility traces, bootstrapping, reproducible random seeds, and experiment aggregation are all visible in the implementation.

The source frames the assignment around the Windy Gridworld problem. The environment contains a 7×10 grid, the start state is $(3, 0)$, the goal state is $(3, 7)$, and the column-wise wind vector is $(0, 0, 0, 1, 1, 1, 2, 2, 1, 0)$. Actions are the four cardinal moves: up, down, left, and right. After a primitive action, the selected column imposes an upward displacement determined by the wind strength, with boundary clipping. Every transition receives a reward of -1, and an episode terminates when the goal state is reached.

The main methodological purpose of the code is comparative rather than architectural: the same tabular task is used to study the effect of backup horizon (n), eligibility-trace parameter (λ), and learning rate (α). This report explains how that comparison is implemented and what the implementation choices imply.

2 Project Overview

At a conceptual level, the program follows this sequence:

1. define a fixed experimental configuration and the Windy Gridworld constants;
2. create filesystem directories for `tables`, `figures`, `logs`, `data`, and the final package;
3. validate basic properties of the environment and serialize an explicit environment specification;
4. train n-step Sarsa and Sarsa(λ) under the main comparison settings;
5. convert episode-level step counts into long-form Pandas records;
6. aggregate those records into summary statistics and learning-curve `tables`;
7. run a smaller sensitivity study for selected values of α;
8. generate plots and configuration artifacts; and
9. package source files and generated artifacts into a ZIP archive.

The source is therefore more than a pair of learning functions. It is an end-to-end research-computing script that combines an environment model, algorithms, experiment orchestration, statistical aggregation, visualization, logging, and distribution of artifacts. The implementation explicitly states that no external dataset is required because Windy Gridworld is an environment specification rather than a real-world data analysis task.

Table 1: Core configuration recovered from the source.

Parameter	Value in source
Grid size	7×10
Start state	$(3, 0)$
Goal state	$(3, 7)$
Wind	$(0, 0, 0, 1, 1, 1, 2, 2, 1, 0)$
Actions	up, down, left, right
Reward per transition	-1
Discount factor γ	0.99
Exploration rate ϵ	0.10
Main learning rate α	0.50
Main episodes	100
Main seeds	$13, 42, 123$
n values	$1, 5, 10$
λ values	$0, 0.1, 0.5, 1$
Alpha-sweep values	$0.1, 0.3, 0.5, 0.7$
Alpha-sweep episodes	80
Alpha-sweep seeds	$13, 42$
Maximum steps per episode	50 000

3 Technical Foundations

3.1 Markov decision process representation

The implementation can be interpreted as a finite Markov decision process with state space $\mathcal{S}$, action space $\mathcal{A}$, deterministic transition function $P(s' \mid s, a)$, and reward function $R(s, a, s') = -1$.

The state space has 70 discrete states and the action space has four actions, so the tabular action-value function contains only $70 \times 4 = 280$ entries.

The source does not instantiate a general probability model. Instead, the transition is deterministic, and stochasticity enters through epsilon-greedy action selection and pseudorandom seeds. This separation is useful: environment dynamics are deterministic while the learning policy remains exploratory.

3.2 Action-value function

The algorithms maintain a table $Q(s, a)$ representing the expected discounted return obtained after taking action a in state s and subsequently following the current policy. The source initializes every table entry to zero. Because rewards are negative, learning drives useful state-action values toward negative numbers, with actions leading efficiently toward the goal receiving less negative returns than actions that prolong the episode.

3.3 Epsilon-greedy policy

For a state s, epsilon-greedy selection chooses a uniformly random action with probability ϵ and a greedy action with probability $1 - \epsilon$. In mathematical form,

$$A_t = \begin{cases} \text{Uniform}(\mathcal{A}), & \text{with probability } \epsilon, \\ \arg\max_{a \in \mathcal{A}} Q(S_t, a), & \text{with probability } 1 - \epsilon. \end{cases} \tag{1}$$

The implementation explicitly detects approximate ties with `np.isclose` and samples among tied greedy actions. However, the accelerated training kernels do not call the separate Python helper `epsilon_greedy_action`; they implement their own compact tie-handling logic. When all four actions are tied, the training kernels choose action 3 (right) rather than applying the helper's more elaborate distance-based tie rule. This is a meaningful implementation detail because it affects the first action selection while the Q-table contains no information.

4 Data and Input Processing

There is no imported dataset. Instead, the experiment is parameterized entirely by a synthetic but fully specified environment. The environment specification is serialized to JSON with grid shape, start and goal coordinates, wind by column, action names, reward, discount factor, and termination rule. This is a useful reproducibility choice because a future reader can reconstruct the task definition from an explicit artifact rather than relying on an implicit description.

State coordinates are converted into compact integer indices via

$$\text{index}(r, c) = r\, N_{\text{cols}} + c, \tag{2}$$

with the inverse operation provided by integer division and remainder. For the 7×10 grid, this produces contiguous indices from 0 through 69. The goal state $(3, 7)$ consequently has index 37, which the environment validation function checks explicitly.

The experimental record is stored in long form. Each row contains the algorithm family, parameter name and value, learning rate, epsilon, gamma, seed, episode number, and steps-to-goal. This design supports grouped statistical aggregation and downstream plotting because algorithm configurations remain explicit columns rather than being encoded in filenames.

5 System Architecture and Code Organization

The source can be divided into six logical layers. First, configuration and filesystem helpers define the experiment contract. Second, the environment and validation code define the task. Third, the learning algorithms implement n-step Sarsa and Sarsa(λ). Fourth, experiment utilities repeatedly execute the algorithms across parameter settings and seeds. Fifth, aggregation and visualization utilities convert raw episode records into summary artifacts. Sixth, packaging and execution functions coordinate the full run and collect outputs.

The main immutable configuration is represented by the frozen `dataclass` ExperimentConfig. This centralizes episode counts, seeds, learning parameters, and the maximum episode length. The use of an immutable configuration object reduces the chance that a later cell or function silently changes an experiment setting.

The environment itself is implemented as a small class with reset and step methods. The learning kernels, however, duplicate the transition logic internally so they can be compiled by Numba. This duplication improves computational locality but creates a maintenance responsibility: environment behavior exists in both the Python class and the compiled algorithm kernels, so changes to one must remain synchronized with the other.

6 Detailed Environment Methodology

The transition procedure is deterministic and proceeds in four conceptual steps. First, the selected cardinal action modifies the row or column. Second, the new coordinates are clipped to the grid boundaries. Third, the wind for the resulting column moves the agent upward by the corresponding integer amount. Fourth, the row is clipped again, the new state is constructed, the reward is assigned, and the goal condition is evaluated.

The wind vector is interpreted by column. If $w(c)$ denotes the wind strength of column c, the vertical component after the action can be expressed as

$$r' = \text{clip}\left(r_{\text{action}} - w(c'),\, 0,\, N_{\text{rows}} - 1\right), \tag{3}$$

where r_{action} and c' are the row and column after the primitive action and clip denotes boundary truncation. Because the wind is applied after the action-induced movement, the order of operations matters.

The validation routine checks the reset state, the goal-state index, a zero-wind transition from column 0 to column 1, and a transition whose target column has wind strength 2. These are small but targeted deterministic tests. They help verify that coordinate indexing and wind application are consistent with the intended task.

7 n-step Sarsa

The n-step Sarsa implementation uses an on-policy multi-step temporal-difference backup. For a visited state-action pair (S_τ, A_τ), the n-step target is the sum of up to n observed discounted rewards plus, when the terminal state has not yet been reached, a bootstrapped estimate:

$$G_{\tau:\tau+n} = \sum_{i=\tau+1}^{\min(\tau+n,T)} \gamma^{i-\tau-1} R_i + \mathbf{1}_{\{\tau+n<T\}} \gamma^n Q(S_{\tau+n}, A_{\tau+n}). \tag{4}$$

The tabular update is then

$$Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha \left[G_{\tau:\tau+n} - Q(S_\tau, A_\tau) \right]. \tag{5}$$

The source implements this logic explicitly with arrays for states, actions, and rewards. A moving time index t defines the state-action pair being executed, while $\tau = t - n + 1$ identifies the oldest pair that has accumulated enough information to receive an update. The target is computed by summing the available rewards and, when appropriate, adding the n-step bootstrap term.

The implementation allocates per-episode arrays whose length is controlled by the configured maximum episode length. This means the memory footprint of the n-step learner scales with the configured maximum horizon rather than with the actual episode length. The conservative bound is simple and predictable, while the upper limit also serves as a guard against an episode becoming trapped indefinitely.

The public wrapper validates $n \geq 1$, $0 < \alpha \leq 1$, and $0 \leq \epsilon \leq 1$. The internal Numba kernel also fixes the random seed for each training run and uses a zero-initialized Q table.

8 Sarsa(λ) and Eligibility Traces

The Sarsa(λ) learner uses a one-step temporal-difference error together with replacing eligibility traces. For a nonterminal transition,

$$\delta_t = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t). \tag{6}$$

At the terminal transition, the bootstrap term is omitted, giving

$$\delta_t = R_{t+1} - Q(S_t, A_t). \tag{7}$$

The trace for the most recently visited state-action pair is then set to one, implementing the replacing-trace rule:

$$e_t(S_t, A_t) \leftarrow 1. \tag{8}$$

The value function is updated over the whole trace table:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \, \delta_t \, e_t(s, a), \tag{9}$$

followed by trace decay,

$$e_t(s, a) \leftarrow \gamma \lambda \, e_t(s, a). \tag{10}$$

The wrapper validates $0 \leq \lambda \leq 1$ in addition to the learning-rate and epsilon constraints.

This implementation is particularly transparent because the eligibility trace is stored as a full array with the same dimensions as the Q-table. Since the environment has only 280 state-action pairs, a dense trace matrix is computationally inexpensive. The same implementation pattern would become much more expensive in a substantially larger tabular problem because every time step performs a full-array update.

8.1 Interpretation of the parameter range

The selected values span the conceptual range from one-step TD behavior toward longer credit assignment. At $\lambda = 0$, the trace decays immediately after the current update, recovering one-step temporal-difference learning. At larger λ, recent state-action pairs retain more eligibility before decaying. At $\lambda = 1$, the trace decay factor becomes γ, so credit persists throughout the episode subject to the discount factor. The code evaluates this family at a fixed main learning rate of 0.5.

9 Optimization and Learning Procedure

The source uses stochastic on-policy learning rather than batch optimization. There is no gradient-based neural network training and no external optimizer such as Adam or L-BFGS. The learning parameters are updated directly through temporal-difference recursions. The learning rate α controls the size of each Q-table correction, while $\gamma = 0.99$ controls the relative influence of delayed rewards and $\epsilon = 0.10$ maintains exploration.

A subtle but important distinction concerns the wording in the source. The source describes the environment as having an undiscounted reward of -1 per time step," which is a statement about the reward signal rather than the discount factor. The actual configuration sets $\gamma = 0.99$. Therefore the implementation uses a negative per-step reward together with discounted return calculations. This is not the same as the strictly undiscounted setting $\gamma = 1$. The report preserves the implemented value because the code, rather than the prose label, determines the actual numerical algorithm.

10 Experiment Execution

The experiment runner abstracts the common logic for both algorithm families through the `run_method` function. It receives a family name, a parameter name and value, an α, a seed sequence, and an episode count. For each seed it invokes the corresponding training function and then emits one record per episode. This provides a consistent data schema across n-step Sarsa and Sarsa(λ).

The main experiment holds $\alpha = 0.50$, $\epsilon = 0.10$, and $\gamma = 0.99$ fixed while comparing all requested n and λ configurations across three seeds. The total number of main training runs is therefore

$$3 \text{ n-step configurations} + 4 \text{ trace configurations} = 7 \tag{11}$$

per seed, or $7 \times 3 = 21$ independent training runs. The code records the resulting episode trajectories at a one-row-per-episode resolution.

The alpha-sensitivity study is intentionally narrower. It compares only $n = 5$ for n-step Sarsa and only $\lambda = 0.5$ for Sarsa(λ), across four learning rates and two seeds. This is a focused probe of α rather than a full factorial sweep over every algorithm configuration. It keeps the computational burden smaller while still exposing learning-rate dependence.

11 Aggregation and Statistical Interpretation

The function `summarize_runs` produces two related outputs. First, it groups by algorithm configuration and episode and computes the mean, median, and standard deviation of steps across seeds. This becomes the learning-curve table. Second, it groups by algorithm configuration and computes summary statistics over the rows belonging to the final analysis window. It records the mean and median steps, standard deviation, best episode, episode count, and number of unique seeds.

The key aggregation detail is the default window value of 100. In the main experiment, there are exactly 100 episodes per run. The selection condition is episode number greater than `max_episode - window`, so every main episode is included. Consequently, the quantity named `mean_final_window_steps` is numerically an all-episode mean for the main experiment, not a late-training-only mean.

The same function is reused for the alpha study, where each run has 80 episodes. Because $80 < 100$, the same condition again retains every episode. Thus the alpha table's nominal final 100-episode interpretation also becomes an all-80-episode average. This is not a coding failure in the aggregation function itself; it is a consequence of applying a fixed window of 100 to runs shorter than that window. It should be interpreted carefully in any academic discussion of the experiment.

A further statistical consideration is that the summary mean is computed over individual episode rows after concatenating seeds. For the main study this corresponds to as many as $3 \times 100 = 300$ episode observations per configuration, and for the alpha study $2 \times 80 = 160$. Those observations are not independent in the classical statistical sense because consecutive episodes within the same run are temporally dependent. Therefore the resulting standard deviation describes dispersion across episode-level observations, not a standard error that should automatically be interpreted as uncertainty across independent experimental replicates. The code does correctly track the number of unique seeds, but it does not compute confidence intervals or seed-level inferential statistics.

12 Implementation Details

12.1 Numba acceleration

Both core learners are decorated with @njit(cache=True). This directs Numba to compile the numerical loops to optimized machine code. The project therefore separates a user-facing Python wrapper, which performs validation, from a compiled numerical kernel, which performs the inner loop. The design is appropriate for a pedagogical tabular RL task because the algorithms contain tight Python-style loops that would otherwise incur significant interpreter overhead.

12.2 Randomness and reproducibility

Each training kernel calls np.random.seed(seed) before creating the Q-table and starting the episode loop. The outer configuration fixes explicit seed lists. The Python helper that seeds the Python and NumPy generators also seeds both the Python standard-library random generator and NumPy, although the main Numba kernels rely on their local NumPy seed. This means the actual stochastic trajectory of the accelerated learner is controlled primarily inside the kernel.

The reproducibility strategy is strengthened by saving the experiment configuration to JSON. The serialized object includes the dataclass parameters plus the environment, n values, and λ values. This is a good research-software practice because the numerical settings can be recovered from the generated artifact rather than reconstructed from source inspection alone.

12.3 Error handling

The source uses explicit validation and runtime checks. Invalid hyperparameters raise ValueError; non-finite Q-values trigger a failure; and episodes exceeding the configured maximum number of steps are rejected. These checks prevent silent corruption and make failures easier to diagnose.

12.4 Filesystem organization

Outputs are partitioned into `tables`, `figures`, `logs`, `data`, and `submission_package` subdirectories beneath an `outputs` root, with a separate `data` directory at the project root. The

organization reflects a sensible separation between raw/structured `data`, visualizations, `logs`, and distributable artifacts.

13 Execution Workflow

The top-level `execute` function calls `ensure_directories`, then `run_all_experiments`, then `create_submission_package`, and finally prints a compact textual summary. The experiment runner first validates the environment and writes the environment specification. It then executes the main comparison, executes the alpha sweep, writes CSV `tables`, produces visualizations, saves the configuration JSON, and returns all in-memory result `tables` in a dictionary.

The final packaging routine creates a clean package directory, copies the source code and audit report when present, optionally copies the notebook, copies generated `tables`, `figures`, `data`, and `logs`, and then creates a ZIP archive. A later notebook section also verifies required artifacts and attempts a Google Colab browser download if the Colab API is available.

One practical caveat is that the uploaded artifact contains notebook-oriented path assumptions and a source-copy fallback that references /mnt/`data/assignment_solution(1).py`. The actual uploaded file name in this review is different. Such path-specific behavior is appropriate for the original runtime context only when the expected file exists; it is not a portable library-level assumption.

14 Design Decisions and Rationale

Several implementation decisions are technically coherent. The small state-action space justifies a dense NumPy Q-table. Explicit integer indexing simplifies memory layout and makes compiled kernels straightforward. Numba reduces the overhead of tight loops. A common `run_method` abstraction avoids duplicating experiment orchestration. Long-form Pandas records simplify aggregation and make the results easy to audit. JSON serialization makes the environment and configuration explicit. Finally, packaging source and artifacts together supports reproducibility and external review.

At the same time, some choices should be interpreted as assignment-oriented engineering rather than universal best practice. The duplicated environment transition logic in the Numba kernels improves performance but weakens single-source-of-truth guarantees. The fixed first action of 3 is a deterministic convention rather than a general epsilon-greedy policy draw. The three-seed main experiment and two-seed alpha study are useful for repeatability but are small as a basis for statistical inference.

15 Computational Considerations

Let $S = |\mathcal{S}| = 70$, $A = |\mathcal{A}| = 4$, and H denote the actual number of steps in an episode. The Q-table storage is $O(SA)$, which here is constant-sized in practical terms.

For n-step Sarsa, each update computes up to n discounted reward contributions. The dominant per-episode work is therefore approximately $O(Hn)$, ignoring constant-size policy and transition operations. Since $n \in \{1, 5, 10\}$ and H is capped, the workload remains modest. The episode buffers require $O(H_{\max})$ memory for states, actions, and rewards.

For Sarsa(λ), the trace matrix has size SA and the update `q_values += alpha * td_error * traces` touches the whole matrix at every time step. The per-episode update cost is therefore

approximately $O(HSA)$ rather than merely $O(H)$. In this specific environment, $SA = 280$, so the cost is still small; in a large state-action space, a sparse-trace representation would be more scalable.

The use of Numba is particularly relevant to the second observation. Compiled dense-array updates are inexpensive for this problem, allowing the simple implementation to remain practical without introducing sparse `data` structures.

16 Reproducibility and Reliability Assessment

The project contains several positive reproducibility mechanisms: explicit seeds, centralized configuration, deterministic environment validation, explicit environment serialization, stable CSV output formatting, and artifact packaging. These features make it easier for another researcher to understand and rerun the intended study.

Nevertheless, reproducibility is not identical to identical execution across every machine. Numba compilation, floating-point implementation details, library versions, and execution environment can influence low-level numerical behavior. The source does not pin package versions in the uploaded file, and it does not encode a full environment specification such as a requirements.txt or lock file. The report therefore characterizes the project as seed-controlled and configuration-recorded rather than fully environment-locked.

A second reliability issue follows from the uploaded file format. The file begins with notebook-export documentation and Markdown-like triple-quoted sections before the future import. A raw Python interpreter rejects this because `from __future__ import annotations` is required to appear near the beginning of the module. This means the supplied artifact should not be described as a clean executable .py module in its current saved state. The underlying implementation logic is still analyzable because the source content clearly exposes the functions and execution pipeline, but a publication repository should store a syntactically clean Python source file.

17 Limitations and Technical Considerations

The most consequential implementation considerations are the following.

- The uploaded .py file has a packaging/syntax issue around the future import. This is a source-format problem, not a flaw in the mathematical definition of Sarsa.
- The environment transition logic is duplicated between the object-oriented environment and the Numba kernels. Divergence between these copies is possible if one changes without the other.
- The helper `epsilon_greedy_action` implements a distance-based tie-break when all actions are tied, but the compiled training kernels instead choose action 3. The training behavior is therefore determined by the compiled kernels, not by the helper's special-case rule.
- The configured discount factor is 0.99, even though the source prose uses the term "undiscounted" in describing the task's reward setup. The numerical learner is discounted.
- The main summary window equals the full 100-episode run, and the alpha-sweep run is shorter than the 100-episode window. Consequently, the named final-window means are whole-run averages for both studies.
- The alpha sweep covers only one n-step configuration and one trace configuration. It is

therefore a focused sensitivity analysis rather than a global study of α across all seven main configurations.

- The number of independent seeds is small, and the summary statistics are computed at the episode-row level. The resulting standard deviations should not be presented as confidence intervals for independent runs.
- The uploaded source does not pin exact dependency versions, so exact binary reproducibility is not guaranteed across environments.

These observations should be treated as implementation-level qualifications rather than as evidence that the assignment solution is conceptually unsound. The underlying algorithms are expressed in recognizable tabular TD-learning form, and the surrounding infrastructure reflects a clear attempt at reproducibility and artifact management.

18 Overall Technical Assessment

From a software-engineering perspective, the project demonstrates strong separation of concerns for a relatively compact research script. Configuration, environment definition, learning kernels, orchestration, aggregation, visualization, and packaging are represented as distinct functions or modules of responsibility. The use of a `dataclass`, explicit validation, Numba kernels, structured result tables, and JSON metadata reflects research-oriented programming discipline.

From an algorithmic perspective, the most important operations are implemented directly rather than delegated to a black-box library. The n-step return is constructed explicitly, bootstrapping is applied only when the n-step target has not reached termination, and Sarsa(λ) uses a concrete replacing-trace update. This makes the source suitable for demonstrating understanding of the temporal-difference mechanisms themselves.

From a scientific-communication perspective, the project is strongest when it reports exactly what the implementation does: the task is explicit, the parameter values are centralized, the seeds are named, and the generated artifacts have stable schemas. The main opportunities for improvement are not conceptual complexity but rigor of experimental interpretation: tightening the summary-window semantics, clarifying the role of γ, aligning helper and kernel policy logic, reducing source-format ambiguity, and documenting the software environment would make the artifact more robust for academic reuse.

19 Conclusion

The uploaded project implements a compact but complete tabular reinforcement-learning workflow for Windy Gridworld, centered on n-step Sarsa and Sarsa(λ). Its principal technical components are the deterministic windy environment, compact state indexing, epsilon-greedy action selection, multi-step Sarsa backups, replacing eligibility traces, seed-controlled execution, Numba-accelerated inner loops, structured Pandas aggregation, and reproducible artifact packaging.

The source is therefore best understood as both an assignment solution and a small research-computing pipeline. Its design exposes the mathematical learning rules directly while also providing the surrounding infrastructure needed to execute a parameterized comparison and preserve the resulting artifacts. The analysis also shows that academic interpretation should follow the exact implementation: the learner uses $\gamma = 0.99$, the main final window equals the full run, and the uploaded .py artifact itself requires a source-format correction before it can be

treated as a clean standalone Python module. These distinctions are important because they separate what the project intends to demonstrate from what the saved file literally executes.

Source Traceability Notes

This report was derived from the complete uploaded source artifact, which contains 942 numbered lines in the supplied file representation. The source explicitly defines the Windy Gridworld constants, the two learning families, the experiment configuration, result aggregation, plotting, and packaging workflow. The report intentionally excludes generated `figures` and numerical result `tables` from presentation, in accordance with the requested deliverable scope.